Chio: A Peer-to-Peer Economy of Verifiable Work

The Chio Project
https://github.com/backbay-labs/chio

October 2026

Abstract

Autonomous agents must coordinate work across systems whose owners retain control of their tools, data and funds. We present Chio, a kernel architecture for peer-to-peer verifiable work. Chio separates agent planning from the authority and durable execution state governing its effects. Each resource owner admits work under locally controlled capabilities, policies and treaties. Linked commitments bind a selected invocation to its delegated resources, receiving authority and conditions of acceptance. A program grows by adding tasks while preserving the commitments of previously authorized work. Recovery retains the original execution history, and separately funded agreements preserve payments earned by accepted work. Signed receipts carry evidence of these decisions between participating domains. Under explicit trust assumptions, we establish preservation of resource bounds, invocation identity and earned claims through plan growth and recovery. Chio's kernel supplies the authority and recovery mechanisms; a fixed-profile adapter composes their work-admission bindings. Evaluation under one administrator exercises plan growth, process loss and settlement. The architecture provides an execution foundation for the agentic web: agents can select collaborators as their plans develop, while each owner enforces the conditions governing its resources and prior commitments.

1 Introduction

An agent program can acquire tasks and collaborators as it runs. It may inspect data held by one organization, delegate analysis to another and purchase a service from a third. Its control flow develops through these interactions, while the resources it uses remain subject to different owners. Executing such a program requires records establishing what each participant may do, what it has already done and which obligations its work has created.

The architectural problem is to preserve the meaning of those records as the plan changes. Delegation of a task, authorization to execute it, evidence of its outcome and acceptance for payment are distinct decisions. A new plan must respect resources already committed. A replacement process must recover the history of earlier actions. An owner must be able to enforce its own conditions when admitting work from another participant.

We present Chio, a kernel architecture for peer-to-peer verifiable work. Chio separates agent planning from the authority and durable state governing its effects. An untrusted agent proposes operations. A receiving kernel admits them under its owner's capabilities, policies and treaties, assigns their execution rights to durable records, and mediates access to local tools. Signed receipts make the kernel's decisions available as evidence to other participants. Each participant selects the authorities whose evidence it accepts.

The unit of composition is a *work commitment*: linked signed records that bind a selected invocation to a bounded allowance, the receiving owner's authorization and, for paid work, its settlement terms.

The allocator, receiver and settlement system each retain the state for the resource they control. Authenticated references connect their decisions to the same invocation. The resulting program spans these authorities without requiring one of them to control every participant's resources.

This organization gives programmable sovereignty an operational meaning. Owners express conditions for cooperation through capabilities, policies and treaties, and enforce those conditions at admission. A foreign agreement is evidence evaluated under that local authority. Once work has been admitted, its execution history remains authoritative through recovery; once a funded claim has earned payment, subsequent work cannot revoke its reserve.

By *verifiable work*, we mean work with checkable authorization, an attributable execution history and acceptance under agreed rules. Verification establishes these facts under the selected authorities and acceptance procedure. The quality of that procedure determines what acceptance says about the result.

The construction separates a task's identity from the version of the plan that contains it. Checked extensions retain earlier allocations and execution rights. Durable admission prevents a new graph signature or a restarted planner from renewing a consumed right. Independently funded agreements preserve earned claims through the failure of the larger program. We specify these mechanisms and establish a conditional preservation result for their composition. The implementation and evaluation follow delegated work through task discovery, graph growth, process loss and settlement, and compare the financial rules with transactional escrow and ERC-8183.

2 Kernel architecture

Chio places an execution boundary between an agent's proposed actions and the resources those actions affect. The host supplies isolation for agents and tool servers. The kernel supplies capability validation, policy and guard evaluation, durable execution records and signed receipts for mediated tool calls. These responsibilities define the trusted runtime interface on which an agent program relies.

2.1 The kernel boundary

An *agent* proposes work; a *resource owner* decides which work may use its tools, data or funds. The owner's capability authority issues bounded grants. A *receiving kernel* checks those grants and the proposed invocation before dispatching to a tool server. Guards evaluate the input and returned output. The kernel retains the invocation's state and produces signed receipts over its decisions, recorded in an append-only receipt log. Host isolation and tool confinement must make the kernel the effective mediator of the protected tools.

Planning remains outside this trusted boundary. An agent may revise its plan, select a provider or request a continuation, but each effectful operation must pass the receiving kernel's checks. The kernel's retained records determine which execution rights have been consumed and what outcomes are known. An agent's account of an earlier call cannot replace those records. Figure 1 shows this separation across two owners.

2.2 Admission and programmable sovereignty

Admission is the receiver's decision to allow a particular invocation. The receiver loads trusted issuers, capabilities, policy, treaties and replay state from stores controlled by its owner. A request

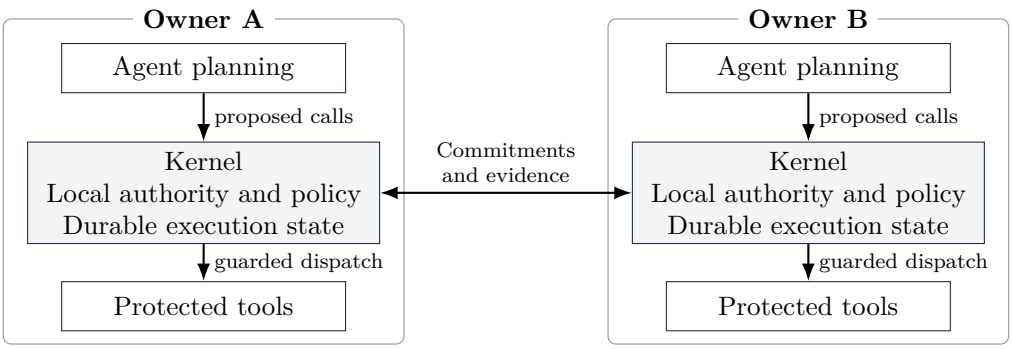


Figure 1: Local enforcement within a distributed program. Each owner controls its kernel's authority and protected state. The interdomain arrow denotes signed records exchanged through the protocol, independently of their transport.

can refer to these authorities but cannot install them. Its signatures and identifiers must agree with the caller, tool, input, resource allocation and receiving kernel actually involved. Before dispatch, the kernel durably associates the invocation with its execution rights and budget hold.

This local control is *programmable sovereignty*: an owner defines the conditions under which external work may affect its resources and enforces them at the execution boundary. Cooperation is expressed through a bilateral treaty. The treaty records the participants, permitted actions, validity period and required invocation and continuation evidence. The receiver verifies the configured participants' signatures over the same statement and checks its conditions together with capability, revocation and guard requirements.

A continuation identifies a right to perform work under the relevant agreement. Its use is recorded so that replay cannot create another invocation. Treaties therefore give remote cooperation a locally enforceable form: another owner's agreement supplies evidence for admission under the receiver's own rules. The receiver retains the authority to refuse a fresh call.

These decisions have different lifetimes. Current policy governs fresh work. An admitted call has an execution history that recovery must consult. An earned escrow claim follows the terms under which its reserve was funded. Changing policy can restrict subsequent actions while preserving the facts and obligations created by earlier ones.

2.3 Program state across owners

An owner may authorize an agent to use part of a resource allocation. That agent is the allocation's *holder*; it can subdivide the allowance or select a provider within its terms. The owner's *allocator* records these choices. A signed task graph describes dependencies, allocations and routes to receiving kernels. A paid task also names a payer, a verifier that judges the result, and an escrow that reserves and transfers funds. One organization may perform several roles.

The program's state is distributed among these authorities. The allocator records assigned allowances; the graph store retains authorized plan versions; the receiver records admitted calls and their outcomes; the escrow records funds reserved, earned, paid or refunded. Work commitments bind these records to the same invocation. An allowance establishes a spending limit, while a deposit supplies funds. A refund records a financial outcome without establishing whether a tool executed.

A *protected* store is durable state controlled by the authority responsible for that resource. All uses of the resource must consult that state. Each delegation root has one protected allocator. Each declared pool for a work graph has one protected sequence of graph versions, whose updates are serialized. Each receiving key has one protected execution and budget domain. The settlement system serializes financial transitions and couples them to actual transfers. Copying a key and database into independently spendable forks, or provisioning two graph sequences against the same declared pool, violates these premises.

Peer-to-peer execution takes place between these locally governed domains. It requires agreement on the commitments for their shared work and exclusive custody of each committed resource. Different programs can use different allocators, receivers and settlement systems. The architecture imposes no common ordering of unrelated work across all participants.

2.4 Trust and verification

Agents and couriers may be malicious. Messages may be duplicated, reordered, delayed or lost; a process may crash between durable steps. Safety assumes unforgeable signatures, collision-resistant canonical digests, valid clock checks and honest enforcement by the selected resource authorities. Tools must be confined sufficiently for their declared actions and permitted readers to describe their effects and disclosures.

A receipt authenticates the issuing kernel's account of a mediated call. A verifier's decision attests satisfaction of the chosen acceptance condition. For paid work, the parties trust this verifier and the rule used to recognize a recorded settlement. The argument relies on these particular authorities; each participant checks their evidence under its own configuration.

The evaluated configuration recruits from previously enrolled principals. Discovering a task permits a new selection within the authorized set; it does not add a trusted issuer or authorize a new reader. Appendix A maps the architecture to the delegation, graph-evolution and funded-work profiles used in the experiments.

3 Delegation and work commitments

A work commitment connects the decisions through which a proposed task acquires permission, resources and an execution identity. The owner allocates an allowance, a provider offers to perform the task, the holder selects that offer, and the allocator freezes the choice. For paid work, the payer reserves funds under an agreement referring to the selected call. The receiver admits the call only when these records agree with its own authorization.

Consider a buyer who funds an API survey and a follow-on review by an intermediary agent. After the survey identifies a problem, the intermediary hires a specialist from its own funds. The specialist completes its task and earns payment, but the intermediary crashes before returning a result to the buyer. The program now contains completed work, an earned claim and an unfinished parent task. Each must retain its identity and terms. In particular, adding the specialist to the plan must not renew the survey's execution right, and failure of the intermediary must not cancel the specialist's payment. We use this example to develop the commitment and its lifecycle.

3.1 Bounded delegation

A work slot specifies a task that may be delegated or assigned to a provider:

$$S = (i, h, E, A, b, c, t, d, Q).$$

Here i is the slot identifier, h its holder, E the permitted tool operations, A the permitted readers, b the spending ceiling in currency c, t the expiry, d the remaining delegation depth and Q the acceptance predicate. Root creation is an owner-controlled operation.

The holder signs a subdivision that names the parent allocation and the complete child slot. The allocator accepts it only if the child's operations and readers are subsets of the parent's, its expiry is no later, its depth is smaller and its currency is unchanged. Child ceilings must satisfy

$$\sum_{j \in \text{children}(S)} b_j \leq b_S.$$

The allocator records the child and updates the parent's assigned amount in one transaction. A slot with children cannot execute, and a selected slot cannot acquire children. Thus the parent and its descendants cannot both spend the same allowance. An internal slot's unused remainder can support later children.

Each allocation is identified by a digest of the allocator's persistent namespace, the root slot and the target slot. This prevents an offer made for one owner's allocation from being reused at another honest allocator that chose the same local names. The child may have its own acceptance predicate: a survey and a specialist review require different checks. The parent task still needs its own result and acceptance.

3.2 Provider selection

The provider signs an offer containing the allocation and slot digests, its receiving key, the tool operation and input, a price ceiling and an expiry. The holder selects the offer together with a request identifier and the digest of a capability issued by the receiver. Until the selection is sealed, a revision check permits replacement while rejecting stale choices.

Sealing is the point after which the choice cannot change. The allocator first freezes the selection, then stores a signed dispatch permit before returning it. The permit contains the slot and signed selection. A receiver can check these records without contacting the allocator at dispatch time; exclusive allocation still depends on the allocator's protected state.

The receiver compares the permit with the caller, receiving key, capability, tool, input and request actually presented. Its capability permits one invocation of that operation with monetary ceilings no greater than the signed offer. Validity, revocation and local policy are checked at admission. Consequently, an offer for one input cannot authorize a different input merely because both calls use the same tool.

3.3 Payment agreements

Paid work adds a payment agreement to the commitment. Figure 2 shows how the execution and settlement authorities use it. The agreement identifies the complete final request, payer, beneficiary, verifier, asset, amount, settlement system and deadlines. Before dispatch, the receiver requires evidence, under its chosen settlement observation rule, that the agreement's allocation holds the

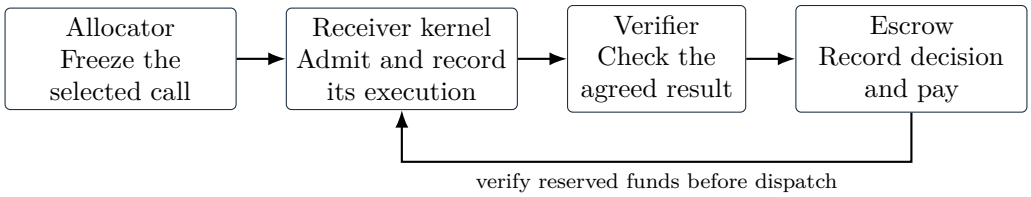


Figure 2: Each authority checks records referring to the same call. The receiver checks local permission and reserved funds before execution. Its evidence feeds the verifier, whose recorded decision governs payment.

required funds. The escrow identifier includes the chain and contract, so identical allocation names on another rail do not identify the same reserve.

A subcontract has its own backing. The intermediary in our example funds the specialist from money it already controls. The payment it expects from the buyer remains locked under a different agreement. The intermediary therefore bears the subcontract cost if its own task fails; the specialist's earned payment does not depend on completion of that task.

3.4 Binding decisions to the invocation

The receiver connects the commitment to the program by checking its signed task graph. The graph identifies tasks and their dependencies, assigns ceilings from the program's declared pool, and carries routes naming the receiving kernels. Its authorization records identify the capabilities permitted for those tasks.

Suppose the specialist's permit and capability authorize receiver R_1, but a correctly signed graph routes the same task to R_2. Both sets of signatures can verify even though the records disagree. The receiver rejects this combination because its own key, the offer's receiving key and the signed route's destination must match. Authenticating each record separately would miss that comparison.

The same principle applies to the other references. The slot, request and graph task must have one task identity. The graph allocation must name the delegated allocation digest and agree on amount and currency. The permit and graph authorization records must name the same capability. These checks establish which invocation the otherwise separate records authorize.

These checks form the common admission contract. A planner can select a new provider or add a task by producing records that satisfy it; the receiving kernel enforces the same relationship between authority, resources and invocation for each such choice. The planner's decomposition of the work can change while this relationship remains fixed.

The payment agreement authenticates the complete request, including its delegation, graph and treaty references. Its financial allocation has a separate identifier linked through that agreement. The references are acyclic: the request contains admission evidence; the agreement commits to the request; the escrow commits to the agreement. Delegation and admission records use canonical JSON and versioned domains [1]; settlement decisions use EIP-712 [2].

The receiver installs these checks from protected configuration before both fresh admission and recovery. Omitting request context or removing local configuration therefore cannot select a path that skips them. The exact signed fields and encodings are given in Appendix A.

4 Execution across evolving plans

The task graph describes the program's authorized structure. Execution records describe what its receiving kernels have admitted and observed. Their identities connect the plan to its history, allowing the graph to grow while earlier work continues under its original commitments.

4.1 Plan extension

Represent the plan as a signed task graph G_i. An extension supplies the digest of G_i and a candidate G_{i+1}. The store verifies both graphs and checks that the candidate retains all earlier tasks, dependencies, routes and allocations. The original policy limits, resource pool and revocation epoch remain fixed. New tasks must fit within the remaining pool and the original depth and fan-out bounds.

Each task has at most one allocation and one single-use continuation. A continuation is signed evidence of an execution right. When the graph grows, an old continuation is signed for the new graph but retains its identifier and all other fields. A new task receives a new identifier. The receiver tracks use by that stable identifier, which makes the distinction between a new signature and a new execution right enforceable.

For example, suppose the survey used continuation c in invocation q. Table 1 shows what happens when the specialist is added. The new graph can describe the survey without authorizing another performance of it.

Record	Before extension	After extension
Survey task and allocation	Original terms	Same terms
Survey continuation	Identifier c, signed for G_i	Identifier c, signed for G_{i+1}
Receiver's execution record	c belongs to invocation q	c still belongs to q
Specialist continuation	Absent	New identifier d

Table 1: Graph versions change; the receiver's record of a consumed execution right does not. A request using the earlier graph retains its earlier signature and resolves that graph from the archive.

Installation compares the expected graph digest, archives the old version and commits the new head in one transaction. Two candidates based on the same head cannot both install. A losing writer must extend the committed successor, which already includes the winning allocation. Historical lookup returns the exact graph named by an old request. Fresh execution under that graph still requires current capability, validity, revocation and treaty checks.

4.2 Admission and execution

The receiver checks the commitment and, for paid work, the observed financial backing, then creates an execution record for the invocation. The treaty and graph continuation rights and the corresponding budget hold are assigned to that record. This state becomes durable before the tool is invoked. A later request cannot acquire a consumed right by presenting a new graph signature or a different request identifier.

The guard checks the returned value against the slot's predicate after output transformations. Paid work also follows the agreement's verification procedure. The execution statement identifies the

original request and output, so the verifier cannot substitute a result from another call without breaking the binding.

For federated work, the statement is exported from the stored admission record. The exporter checks the original treaty, participants, request and admission time against the retained outcome. This authenticates the receiving kernel's execution record. Delivery of a final bilateral receipt may still require a remote co-signer.

4.3 Recovery

A timeout describes a missing response. It does not establish whether the receiver dispatched, whether the tool had an effect, or whether acceptance was recorded. Recovery consults the state held by the authority responsible for each of these facts.

Before sealing, the holder may choose another provider. After sealing, the allocation remains bound to the original request even if a crash prevents delivery of its permit. A stored permit is returned with its original signature and issuance time. If the receiver has recorded a completed result, it can return that result without calling the tool again. If dispatch occurred but authoritative completion evidence is absent, the state remains OutcomeUnknownAfterDispatch. Independent siblings proceed under their own allocations.

Settlement recovery follows the same discipline. The implementation stores the exact signed transaction before broadcasting it. After a restart it observes or rebroadcasts those bytes, preserving the nonce and allocation. This prevents recovery from issuing a different transaction in place of an uncertain one. Recovery of an arbitrary tool effect still depends on that tool's effect model.

4.4 Acceptance and payment

A submitted claim follows one of two paths:

$$\mathsf{Funded} \to \mathsf{Submitted} \to \begin{cases} \mathsf{Payable} \to \mathsf{Paid}, \\ \mathsf{Rejected} \to \mathsf{Refunded}. \end{cases}$$

Undecided work can also expire and refund under its agreed deadlines.

The selected verifier signs a decision identifying the allocation, agreement, submission, beneficiary, asset and amount. The escrow authenticates the decision and records acceptance under the chosen settlement observation rule. That recorded acceptance makes the claim *earned*. A result or an unrecorded signature alone does not. Repeating the same decision has no effect; a conflicting replay fails.

An earned claim has no timeout or refund transition. Its beneficiary can withdraw without another authorization from the payer. Since the parent and child have separate reserves, refunding the parent leaves the child's earned claim unchanged. In the API example, the specialist collects its reserved 100 units even if the buyer is refunded for the intermediary's incomplete task. The refund does not resolve uncertainty about the intermediary's tool effects.

For the funded example, the full order is: seal the call, bind its payment agreement, deposit backing, install the graph extension, admit and dispatch, then record acceptance. No transaction spans this sequence. A crash may strand capacity or funds until the existing agreement resolves them. Retaining that commitment prevents a missing response from being treated as permission to allocate the resource again.

5 Preservation of commitments

The kernel contract must preserve two quantities and one identity: the capacity allocated to work, the funds reserved for payment, and the invocation to which each execution right belongs. We first state the accounting invariants, then show why plan growth and recovery preserve them together.

This is the property a planner needs when extending a distributed program: the extension must account for earlier commitments even when their execution or settlement is incomplete. The following argument concerns the specified composition rules and the authorities that enforce them.

5.1 Delegated and planned capacity

Let B_D be a delegation root's ceiling, b_v a leaf's ceiling and U_v an internal slot's unused remainder. The allocation tree satisfies

$$\sum_{\text{leaves } v} b_v + \sum_{\text{internal } v} U_v = B_D, \qquad b_v, U_v \geq 0. \tag{1}$$

Subdivision moves capacity from a parent into its children. Only leaves can execute, so internal capacity cannot also be spent as a tool call. Selection and sealing change no ceiling. Restrictions on operations and readers, expiry and delegation depth hold transitively along the tree. The receiver's one-invocation capability and charge checks bound each executed leaf.

For a graph G_i, let $\mathcal{A}_i$ be its set of uniquely identified allocations and $b(a)$ the ceiling of allocation a. A checked extension preserves

$$\mathcal{A}_i \subseteq \mathcal{A}_{i+1}, \quad b_{i+1}(a) = b_i(a)\ (a \in \mathcal{A}_i), \quad \sum_{a \in \mathcal{A}_{i+1}} b(a) \leq B_G, \tag{2}$$

where B_G is the original declared pool. One protected sequence of graph versions per pool, with serialized installation, gives this bound a single accounting domain. Since allocations are retained, $\bigcup_{i=0}^{n} \mathcal{A}_i = \mathcal{A}_n$. The latest bound therefore covers every allocation installed, including work still running under an older graph.

5.2 Deposited funds

Consider one deposited amount D_u, accounted separately from later deposits. Let F_u be its available funds, L_u its locked funds, and P_u, R_u its paid and refunded outflows. Then

$$D_u = F_u + L_u + P_u + R_u, \qquad F_u, L_u, P_u, R_u \geq 0. \tag{3}$$

Reservation moves funds from F_u to L_u. Payment or refund moves an allocation's locked funds once to the corresponding outflow. Refunded cash remains an outflow from this deposited amount even if deposited again. A new deposit creates another accounting entry and leaves outstanding locks in the earlier entry unchanged.

Acceptance restricts a claim to $\{\mathsf{Payable}, \mathsf{Paid}\}$. A transition on another allocation cannot change that claim or release its reserve. This is the financial reason that a parent refund cannot revoke a child's earned payment. Withdrawal also requires the beneficiary's key, available evidence, transaction inclusion and a functioning asset.

5.3 The combined guarantee

Proposition 1 (Preservation under plan growth and recovery)**.** *Assume the protected allocation, graph, execution and settlement state of Section 2, the authenticated references of Section 3, and the transitions of Section 4. Start with validly provisioned roots, verified initial graphs and funded accounts. Every finite interleaving of valid subdivision, selection, sealing, graph extension, execution, recovery and settlement has these properties:*

1. *Delegation and graph allocations remain within their original bounds; paid plus refunded funds do not exceed deposited backing.*
2. *A sealed invocation retains its receiver, capability, input, tool operation and request. Plan growth and replay grant it no further execution right.*
3. *An earned child claim retains its terms and remains payable or paid through parent failure, completion or refund.*

Proof. Initially, a root has all its capacity in one leaf, a verified graph fits its declared pool, and a fresh deposit has $(F_u, L_u, P_u, R_u) = (D_u, 0, 0, 0)$. No execution right has been consumed or claim earned. The properties hold in this state. Induct over the permitted transitions.

Subdivision preserves (1) by transferring capacity from the parent remainder to a child. Selection does not alter capacity. Sealing freezes the selected invocation before its permit can leave the allocator.

Graph extension retains all earlier allocations and checks (2) before installing the successor. Existing continuation identifiers remain unchanged. A receiver's record of their use is therefore unchanged as well. The archive supplies the original graph and signature for an old request. Concurrent installers serialize through the same protected head.

Admission checks that the permit, capability, task, route and treaty refer to the presented invocation. The payment agreement commits to that complete request. A substituted object fails a reference check or belongs to a different authorized commitment. Before dispatch, the receiver durably assigns the execution rights and budget hold to the original execution record. Replay consults that assignment; it cannot acquire the same rights for another invocation. Expiry, revocation and policy refusal may prevent a call but cannot create a replacement right.

Recovery returns a stored permit or completed output when available. Otherwise, an unknown execution outcome remains unknown and supplies no fresh authority to call the tool. These steps create no allocation and release no consumed execution right. Financial retries reuse the persisted transaction and its allocation, subject to the same settlement checks as the original submission.

Reservation, payment and refund preserve (3) and affect only their named allocations. Acceptance leaves the child in $\{\mathsf{Payable}, \mathsf{Paid}\}$ under every subsequent child transition. A parent transition changes another allocation.

These cases remain valid when interleaved: each shared resource has a serializing authority, and graph extension retains the resources named by earlier work. The authenticated references require each authority's decision to concern the same invocation, completing the induction. □

The argument depends on exclusive custody at each scarce resource. Two receivers can each be shown a signed promise against one deposit while neither sees the other promise. Each local view is consistent with an affordable exchange. A protected allocator or ledger must establish that both promises have not spent the same capacity; signatures alone cannot do so.

The guarantee is safety under the specified transitions. A missing later record may prevent progress, but it cannot undo an earlier commitment. This is why separate local transactions suffice even when a handoff fails partway through.

The accompanying Lean development checks abstract financial conservation and earned-state lemmas. Tests exercise the concrete admission and recovery rules. The full argument above is conditional on those rules; the artifact does not contain a machine-checked refinement from the argument to the Rust and Solidity implementations.

6 Implementation and evaluation

Chio implements capability evaluation, guarded tool dispatch and receipt signing in its Rust runtime. Delegation allocators and protected stores retain resource assignments; the swarm authority verifier checks task graphs and continuations; treaty admission governs cooperation between receivers. The work-commitment construction binds these mechanisms to durable execution records and funding. In the evaluated implementation, the example adapter's `Bindings` guard joins task identity, allocation digest and amount, receiver identity and request context. Its current program and allocation dimension are fixed, with a 100-unit parent contract. These cross-record joins have not yet been promoted into the public runtime admission API. The W1 runtime plan specifies that generalization; the present results qualify the fixed profile. The concrete profiles and their encodings are specified in Appendix A.

Evaluation examines three consequences of this architecture: rejection of inconsistent commitments at admission, continuity of execution rights through plan growth and recovery, and preservation of earned payment through parent failure. The composed run follows the API example through all three boundaries.

All runs use author-controlled infrastructure. The composed execution has three receiving keys and separate stores on one host, in-process treaty co-signers, a private development chain and mock tokens. The artifact records the source and command output for each run. Earlier financial-model and comparison results retain their original source revisions; Appendix B separates them from the current implementation's qualification.

6.1 Growth followed by parent failure

The API program begins with a declared ceiling of 300 units and three graph nodes: a planning root, a survey and an unstarted intermediary task. The buyer funds the survey and intermediary tasks under separate agreements. The survey has its own receiving key and execution store; its agreement names the intermediary as payment beneficiary. The survey examines an OpenAPI document and finds an operation without required authentication. This returned result triggers selection of an enrolled specialist. Its task, capability, sealed selection, payment agreement and 100-unit deposit are created at that point. The checked graph grows to four nodes while preserving the survey's original execution evidence.

The intermediary calls the specialist. A separately implemented Python checker accepts the bound result, and the escrow records the specialist as Payable. The intermediary process is then killed with SIGKILL before its own return is captured. Its funding eventually refunds, while its execution outcome remains unknown. With parent and verifier settlement signing disabled, the specialist's collector withdraws the original claim. Repeating collection produces neither another call nor another transfer.

Task	Execution outcome	Financial outcome
Survey	Completed; original evidence retained	Paid 100
Intermediary	Unknown after dispatch	Refunded 100
Specialist	Completed; original claim retained	Paid 100

Table 2: Recorded outcomes after graph growth, intermediary process loss and collection. The intermediary's refund does not establish whether its tool completed.

Each receiver records one tool call and one acquisition of its treaty and graph continuation resources. The operation, allocation, budget hold, authorization and durable claim identities remain unchanged through reopening and collection. Of the initial 2,000 mock units, the final balances are buyer 900, intermediary 1,000, specialist 100 and escrow zero. The intermediary receives the survey payment and separately funds the specialist.

The first collection report records an unavailable remote receipt co-signer. A later open has no unfinished local operation to reconcile; both reports retain the same execution and payment identities. The run establishes local completion and payment. It does not establish delivery of a final bilateral receipt.

6.2 Substitution, replay and concurrent growth

The rejection tests present authenticated but inconsistent records. A re-signed graph that changes an issued continuation is rejected. A sealed allocation cannot select a different invocation. Missing treaty evidence, a capability from another receiver, and a signed route naming another receiver are rejected before the funding check or tool execution. The example adapter supplies the missing-treaty and foreign-capability prechecks and the cross-record route join; these cases do not establish the same guarantees for arbitrary callers of the shipped runtime hook. Removing the adapter's route check causes its regression test to fail by reaching the funding check.

Recovery tests return recorded output after process loss, allow independent siblings to proceed, and preserve unfinished work through a graph extension. A fresh request using a consumed continuation is rejected. When two extensions compete for the same graph head, only one commits. These cases exercise the state and reference checks used in the preservation argument.

The current qualification records 21 terminal commands, including the composed run and four existing child-payment crash and transaction-recovery cases. Six optional chain regressions are exercised separately. Exact counts, ignored cases and retained initial failures are recorded in Appendix B and the artifact companion.

6.3 Financial comparison

An implementation of the financial state machine using ordinary SQL transactions matches every compared state and balance on the retained representative traces. The pinned ERC-8183 implementation also pays a separately funded child after parent refund and rejects overcommitment of available reserves. It supports partial settlement in addition to the fixed-price cases considered here. Chio records Payable before withdrawal; the pinned upstream completion pays immediately.

These comparisons establish substantial overlap in the financial mechanisms. The composed run evaluates their connection to the kernel's admission contract, the authorized invocation and its

changing task graph. The authentication inventory provides an easily checked workload for this purpose. It does not measure an economic advantage in outsourcing API review.

7 Related work

Object capabilities provide a basis for cooperation among mutually suspicious components [3]. Proof-carrying code makes supplied evidence checkable against a consumer's policy [4], and Secure Linking connects certified component properties with proof-carrying authentication [5]. Chio uses the receiver's policy in the same role: a remote signature is checked under locally selected authority. Its kernel contract applies this principle across records owned by the allocator, executing kernel and settlement system, all referring to one work commitment. Programmable sovereignty describes the owner's control over that admission boundary.

Contract Net organizes task allocation through announcements, bids and awards [6]. Agent Contracts combines task specifications, resource bounds and success conditions, including delegated-budget conservation [7]. Chio follows selection through to the exact receiver-issued capability and invocation, and defines how that selection survives extension of the signed task graph. Interface Automata and multiparty session types address compatibility and communication properties within their models [8, 9]. The property studied here concerns resource bounds and retained execution rights; correctness of task decomposition requires properties of the tasks themselves.

AgentBound supplies deterministic governance and verifiable decision receipts [10]. A2A provides task, message, artifact and discovery interfaces [11]; AP2 specifies signed commercial and payment mandates [12]. OAuth token exchange leaves trust and token semantics to deployment profiles [13]. Such protocols can carry application-specific agreements. Chio specifies the reference checks and durable state transitions that make those agreements govern the same invocation through execution and recovery.

Sagas describe compensation for sequences of transactions [14]. Durable workflows and transactional outboxes likewise retain work and messages across failures. Chio uses durable records to distinguish financial compensation from evidence about execution: refunding a request does not establish that its tool had no effect. This distinction matters when a recovered planner decides whether another action is authorized.

ERC-8183 is the closest economic construction, with escrowed jobs, providers, evaluator-approved completion and expiry refunds [15]. Section 6 and Appendix B compare the pinned implementations under matched capital and evaluator assumptions. ERC-8004 supplies discovery, reputation and validation interfaces [16] that can inform a receiver's policy. Contingent payments and fair exchange provide cryptographic constructions for particular verification and exchange problems [17, 18]. The funded profile here uses an agreed verifier and settlement system. Its conservation rules also admit a conventional transactional implementation.

Bitcoin resolves competing spends through a shared history [19]. Chio assigns an explicit custodian to each allocation, execution right and financial reserve. Its composition argument assumes that these custodians prevent conflicting uses of their resources. It requires agreement on the records for a particular task, without imposing one history on all programs.

8 Scope

The preservation result applies to the composition rules specified in this paper. Plan extension is additive: old tasks, allocations, permissions and continuation identifiers remain in place. Removing tasks, reclaiming their budgets, migrating receivers or changing the graph's policy limits would require closure or fencing rules beyond this operation. A timeout cannot justify reclaiming a resource whose use is still uncertain. Retaining earlier graph versions also incurs storage and verification costs.

Acceptance is as strong as the agreed check. The implemented delegation predicates test bounded properties of JSON output; the funded verifier can apply a richer procedure. Success of a child does not establish success of its parent. The host must mediate declared operations and readers, and revocation cannot recall information already disclosed.

The safety result permits unavailable or unfinished work. Sealing can strand capacity; contract expiry can prevent output or recovery checks from completing. Hashes detect altered evidence but cannot retrieve withheld evidence. An earned claim preserves the right to withdraw, while collection still requires an available settlement system and an eligible beneficiary. Verifier failure before acceptance is recorded can leave completed work unpaid.

Deployment must protect the authorities in Section 2. A compromised allocator can overissue; forked execution stores can duplicate authority; a dishonest verifier can approve bad output. The asset and settlement observation rule must support the claimed financial transitions. The local experiments establish the reported behavior under their stated configuration. They do not measure independent administration, public-chain finality, hostile-load scalability, integration savings or useful-work profitability.

9 Conclusion

Chio provides a kernel architecture for programs whose work develops across independently controlled resources. Agents determine what to attempt next. Receiving kernels determine what may execute, retain its history and issue evidence under their owners' authority. Capabilities and treaties give this cooperation explicit terms; work commitments bind those terms to particular invocations and their resources.

The construction preserves these bindings as the task graph grows. A later plan cannot renew a consumed execution right, recovery consults the original execution record, and an earned payment retains its separately funded reserve. Under the stated assumptions, these rules preserve resource bounds, invocation identity and earned claims through the specified transitions.

This is the execution model Chio offers the agentic web. A program can discover tasks and select collaborators during execution while each participant retains control over admission to its domain. Planning can be distributed and revised; the commitments governing its effects remain in the custody of the authorities responsible for enforcing them.

References

[1] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, 2020. URL https://www.rfc-editor.org/rfc/rfc8785.

[2] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed Structured Data Hashing and Signing, 2017. URL https://eips.ethereum.org/EIPS/eip-712.

[3] Mark Samuel Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006. URL https://worrydream.com/refs/Miller_2006_-_Robust_Composition.pdf.

[4] George C. Necula. Proof-Carrying Code. In *Proceedings of POPL*, pages 106–119, 1997. URL https://homes.cs.washington.edu/~mernst/teaching/6.893/readings/necula-popl97.pdf.

[5] Eunyoung Lee and Andrew W. Appel. Secure Linking: a Framework for Trusted Software Components (Extended Version). Technical Report TR-663-02, Princeton University, 2002. URL https://www.cs.princeton.edu/techreports/2002/663.pdf.

[6] Reid G. Smith. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980. doi: 10.1109/TC.1980.1675516.

[7] Qing Ye and Jing Tan. Agent Contracts: A Formal Framework for Resource-Bounded Autonomous AI Systems. arXiv:2601.08815v3, 2026. URL https://arxiv.org/html/2601.08815v3.

[8] Luca de Alfaro and Thomas A. Henzinger. Interface Automata. In *Proceedings of ESEC/FSE*, pages 109–120, 2001. URL https://web.cs.wpi.edu/~heineman/html/teaching_/CS562/p109-de_alfaro.pdf.

[9] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty Asynchronous Session Types. In *Proceedings of POPL*, pages 273–284, 2008. doi: 10.1145/1328897.1328472. URL https://mrg.cs.ox.ac.uk/publications/multiparty-asynchronous-session-types/.

[10] Anuj Kaul, Qianlong Lan, and Pranay Gupta. Behavioral Governance for Autonomous AI Agents: The AgentBound Framework. arXiv:2606.30970v2, 2026. URL https://arxiv.org/html/2606.30970v2.

[11] A2A Project. Agent2Agent Protocol Specification, 2026. URL https://github.com/a2aproject/A2A/blob/65dadbd9b900fe2d970e05416eb432a91347cd2a/docs/specification.md. Source pin 65dadbd9b900fe2d970e05416eb432a91347cd2a.

[12] AP2 Contributors. Agentic Payment Protocol, version 0.2, 2026. URL https://ap2-protocol.org/ap2/specification/. Source pin and retrieval hash in the artifact's sources.json.

[13] Michael B. Jones, Anthony Nadalin, Brian Campbell, John Bradley, and Chuck Mortimore. OAuth 2.0 Token Exchange. RFC 8693, 2020. URL https://www.rfc-editor.org/rfc/rfc8693.

[14] Hector Garcia-Molina and Kenneth Salem. Sagas. Technical Report CS-TR-070-87, Princeton University, 1987. URL https://www.cs.princeton.edu/techreports/1987/070.pdf.

[15] Davide Crapis, Bryan Lim, Tay Weixiong, and Chooi Zuhwa. ERC-8183: Agentic Commerce, 2026. URL https://eips.ethereum.org/EIPS/eip-8183. Draft, source snapshot 365b4c02879f3e882b91281d42b4f57b406205e9.

[16] Marco De Rossi, Davide Crapis, Jordan Ellis, and Erik Reppel. ERC-8004: Trustless Agents, 2025. URL https://eips.ethereum.org/EIPS/eip-8004. Draft, source snapshot 365b4c02879f3e882b91281d42b4f57b406205e9.

[17] Matteo Campanelli, Rosario Gennaro, Steven Goldfeder, and Luca Nizzardo. Zero-Knowledge Contingent Payments Revisited: Attacks and Payments for Services. Cryptology ePrint Archive, 2017/566, 2017. URL https://eprint.iacr.org/2017/566.

[18] Stefan Dziembowski, Lisa Eckey, and Sebastian Faust. FairSwap: How to Fairly Exchange Digital Goods. Cryptology ePrint Archive, 2018/740, 2018. URL https://eprint.iacr.org/2018/740.

[19] Satoshi Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. URL https://bitcoin.org/bitcoin.pdf.

A Reference profiles and encodings

The artifact names three existing profiles: D1 for dynamic delegation, S1 for additive swarm evolution and F1 for funded work. The composed receiver joins their original records; it does not replace them with another wire format.

A.1 Delegation records

A D1 allocation digest binds the persistent allocator namespace, immutable root slot and target slot. Each holder-signed subdivision binds the parent's allocation digest. The receiver's offer binds the allocation, complete slot, receiver, effect, input, price ceiling and expiry. The holder's selection includes the complete signed offer, capability digest, request ID and expected selection revision. The allocator-signed permit carries the frozen slot and selection. Signed domains are versioned; the qualified records use v2.

The original receiver envelope contains a slot ID, payload and permit. Its input digest binds the slot ID and exact payload. In the composed layout, the permit is carried in the governed request context and the payload is the entire original tool-argument object. The receiver selects the installed layout; the caller cannot downgrade it. The enclosing permit has its own signature and is excluded from the input digest to avoid a hash cycle.

Canonical numbers are bounded by $2^{53} - 1$, delegation depth by 32, children per slot by 64, slots per root by 4,096 and envelopes by 64 KiB. Acceptance is a conjunction of 1–16 JSON-pointer equalities or inclusive integer ranges. The one-invocation native capability is issued by the receiving kernel and names the exact subject, server and tool. Per-call and total monetary ceilings must be no greater than the signed offer, in the same currency.

The D1 offer specifies accepted-output pricing. The installed guard rechecks the binding before dispatch and validates the exact JSON value after output transformations. A qualifying predicate rejection withholds output and uses the qualified reversible-hold path for a zero-charge denial. The executed invocation remains consumed. Native capability validity and revocation remain independent requirements.

Sealing freezes the selection, then persists its signed permit before returning it. A crash between those durable steps can strand capacity. When a stored permit is available, a retry returns its original issuance time and signature. There is no post-seal replacement or automatic reclaim operation.

A.2 Swarm and receiver records

S1 retains the graph envelope, existing nodes, edges, joins, routes, witness chains, allocations, pool and revocation epoch. Existing continuations retain every field except the new graph digest and signature. New tokens and allocations name new tasks; each task has at most one of each.

The SQLite installer verifies both bundles at store-owned time and commits the head transition and parent archive together. Historical lookup checks the graph ID, signed graph digest, full bundle digest and strict encoding. Other stores need equivalent protected serialization and historical binding to implement the same profile.

The composed receiver pins its allocator, witness, treaty peer, program and runtime profile in protected deployment policy. Each open installs the runtime hook and delegation guard before reconciliation. The existing runtime database must remain present; losing it cannot provision an

empty continuation domain. The funded journal's policy hash prevents removal of the composed configuration.

Bilateral admission uses the existing signed treaty, permission antecedents, invocation and continuation records. Such antecedents establish permission; completed-work evidence comes from the retained native outcome. Execution export revalidates that outcome and its frozen federation provenance under the original participant identities and admission time. It grants no new dispatch, settlement or remote-completion authority.

A.3 Funding and deadlines

F1's native agreement authenticates the full final request, original receiver authority, checker context and settlement terms. The current workload uses 100 mock units per native agreement. The artifact-only F1 agreement is a different concrete schema; the companion maps both explicitly.

Let $t_s < t_c < t_r < t_f$ be submission, challenge, resolution and refund boundaries. Submission is allowed by t_s, first verifier decision recording when $t_c < t \leq t_r$, and undecided refund when $t > t_f$. The interval before decision is a timing boundary; the profile has no public challenge court. An available failed check yields rejection; missing required output or custody yields no eligible decision.

Acceptance is a domain-bound EIP-712 decision over the allocation, agreement, submission, decision digest, acceptance bit, beneficiary, asset and amount. The reference escrow has no later expiry for an earned withdrawal. Its administrative pause and verifier-registry updates govern new funding, without rewriting an existing claim. This fixed-price profile has no partial-settlement entrypoint. Fees, disclosure loss and opportunity costs fall outside its reserved-unit bound unless separately provisioned.

B Artifact and comparison details

The companion `ARTIFACT.md` gives executable commands, source pins, dependency locks and the mapping from protocol objects to implementation. The claim register records assumptions and counterexamples for each claim. Current native evidence and historical experiments retain separate source inventories. The PDF build verifies the derived results and frozen artifacts.

B.1 Current native qualification

The current campaign records 21 terminal commands, 37,341 unchanged source inputs and 48 hashed outputs. Rust pass counts by boundary are 55 workflow, 16 native delegation labels, 5 existing native composition, 476 swarm/runtime, 11 execution-export, 29 durable SQLite and 96 default standalone cases. The delegation count includes its subprocess worker. The workflow run ignores 1 doctests. The default standalone run ignores 6 opt-in chain cases. The separately retained 6-case chain campaign belongs to source inventory `865a331ba3a022f8`, whose full SHA-256 is recorded in `chain-regressions.json`. It is historical evidence and is not added to the current default-suite count.

The campaign also includes Node wire/pin tests, artifact tests, the evolving funded execution, four existing earned-child crash subcases, four strict Clippy commands, three format commands, the delegation example and the funded binary build. This qualifies the affected local boundaries, not the full workspace or a production deployment.

The suite outputs identify the tested bindings and recovery cuts. Intended negative controls and initial development failures remain distinct from the terminal passing results.

B.2 Retained financial model and implementation checks

Experiment	Result	Scope
Finite claim graph	5,508 states	Two prefunded edges and nine timestamps
Transition checks	132,192	Selected role, deadline and financial transitions
Contract suite	143 passing	Private-chain bytecode; 118 representative traces
SQL comparator	312 matched steps	Same representative traces; trusted sequential store
Upstream ERC-8183	76 passing	Unmodified suite at the pinned source
Paired escrow cases	20 passing	Same asset, fees, capital and evaluator trust
Lean specification	7 results	Abstract conservation and earned-state lemmas

Table 3: Historical results retain their original source identities. Their counts are not attributed to the changed native implementation.

The finite graph begins with a three-unit parent and a two-unit child. It explores submissions, decisions, payment, refund, expiry and execution uncertainty within that alphabet. The Solidity runner replays 118 representative financial traces, rather than all 132,192 transitions. It compares state, commitments, decisions and actual balances. A deliberately weakened model permits an earned claim to refund; the explorer finds the counterexample. An in-memory mutation of the escrow produces the corresponding intended test failure.

A subsequent source-bound review campaign replays 270 representatives against the current escrow bytecode, including refund and expiry attempts in both Payable and Paid after the deadline. Every rejection checks its exact contract error. Its 271 bytecode tests pass, and SQL matches 738 non-time steps. The expire-payable mutation produces 9 intended failures. The record in `evidence/claim-replay-20261005` preserves these commands and source hashes separately from the historical table above.

The Lean specification proves conservation over finite account traces, paid-plus-refunded bounds, persistence of earned states, and noninterference between distinct allocation entries. The native binding and graph-growth arguments are not mechanized refinements of the Rust code. These different forms of evidence support their stated boundaries rather than a proof of the implementation as a whole.

B.3 Matched alternatives

The ordinary SQL comparator implements the same monetary transitions without Chio kernel code. It matches 312 non-time steps over the 118 representative traces, with additional tests for child collection, allocation exclusivity and unknown execution. A conditional-backing experiment also compares a structural bound with an independent exhaustive oracle over 4,095 finite profiles. A conventional allocator obtains the same capital saving from selecting among exclusive alternatives before dispatch.

Upstream ERC-8183 is pinned to commit `142e669c1f`. Its 76 unchanged tests and 20 added comparison tests use equal mock assets, zero fees, capital and evaluator trust. Both implementations pay a separately funded child after parent refund, reject two 60-unit jobs funded from 100, and deny

spending an unsettled parent reserve. The upstream implementation also pays an approved 60-unit partial claim from a 100-unit job and refunds the remaining 40 after rejection.

The lifecycle differences are concrete. Chio records acceptance before later withdrawal, applies a resolution cutoff, and preserves earned withdrawal through its funding pause or allowlist changes. The pinned upstream completion pays immediately; after its expiry grace period, completion can still win if refund has not executed. Its refunds are blocked while paused. These observations concern the pinned implementations, whose specifications and extension mechanisms admit other choices. No performance or security ranking is inferred.

The historical security-integration record binds its native runs to their earlier v2/v3 source inventories. Current composed qualification remains separate.